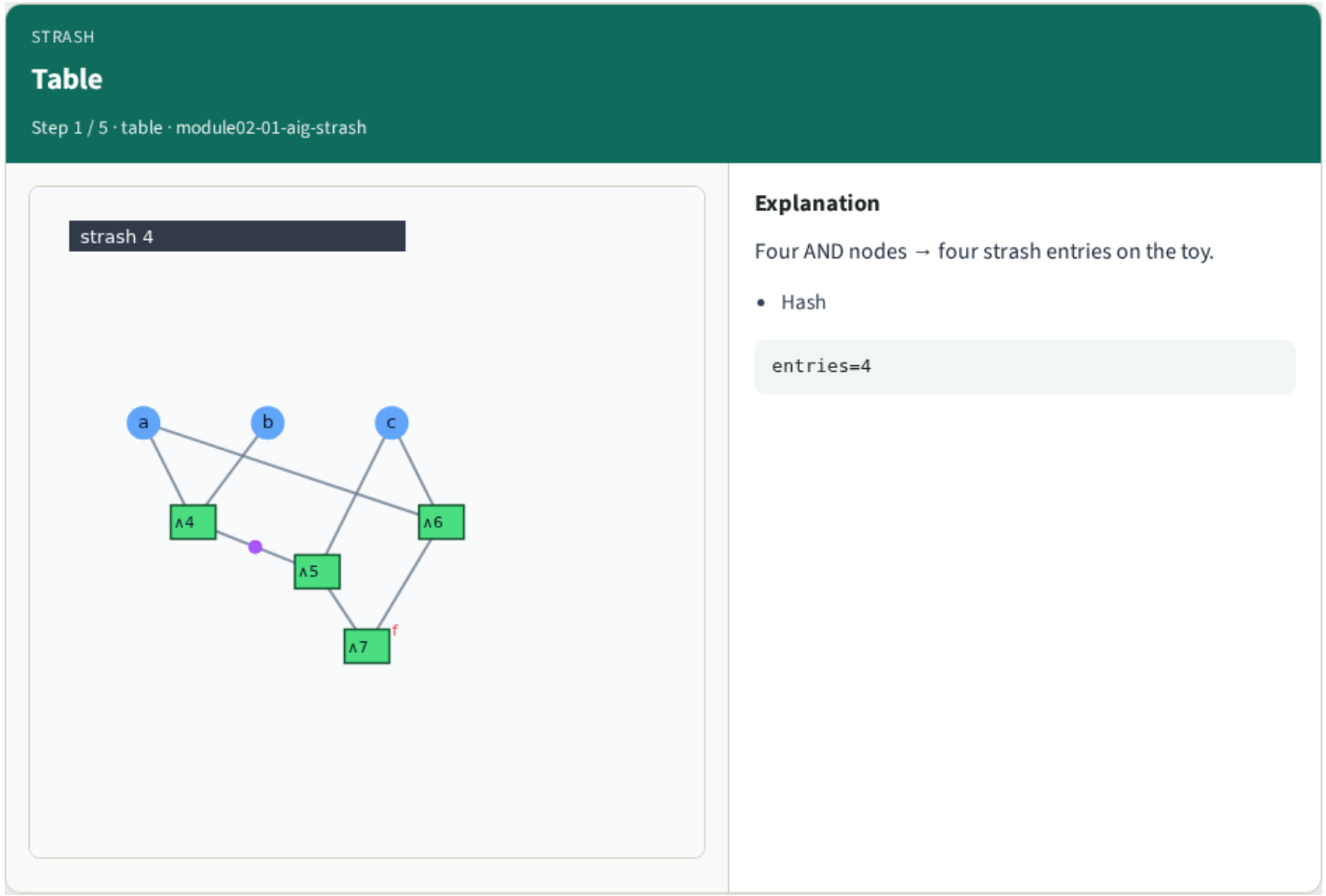
Strash

Strash reuses identical AND structures so the graph stays compact

The idea

- Canonical key sorts the two literals; table maps key → node id

Table


STRASH
Table
Step 1 / 5 · table · module02-01-aig-strash
strash 4
a
b
c
∧4
∧6
∧5
∧7
f
Explanation
Four AND nodes → four strash entries on the toy.
Hash
entries=4

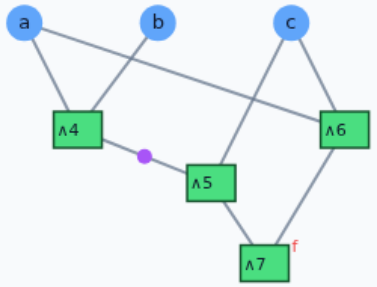
Table

Key

STRASH

Key

Step 2 / 5 · key · module02-01-aig-strash



Explanation

Key orders fanin literals so AND(a,b)=AND(b,a).

- Canonical

```
Sorted lits
```

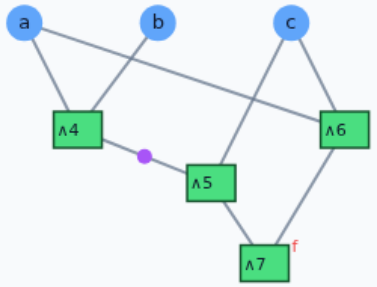
Key

Lookup

STRASH

Lookup

Step 3 / 5 · lookup · module02-01-aig-strash



Explanation

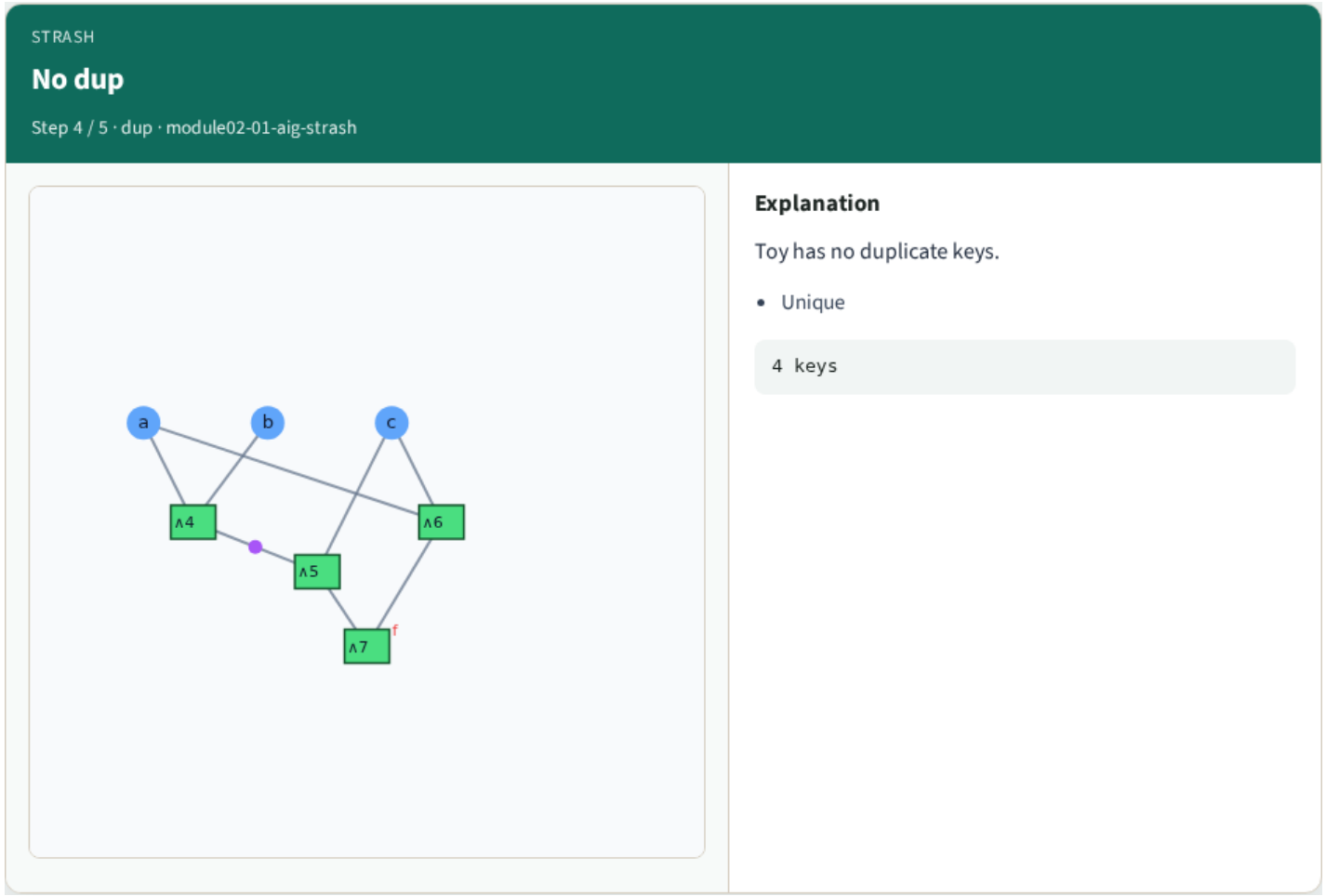
Identical fanins would hit the same entry.

- Reuse

```
No dup AND
```

Lookup

No dup


STRASH
No dup
Step 4 / 5 · dup · module02-01-aig-strash
a
b
c
∧4
∧6
∧5
∧7
f
Explanation
Toy has no duplicate keys.
Unique
4 keys

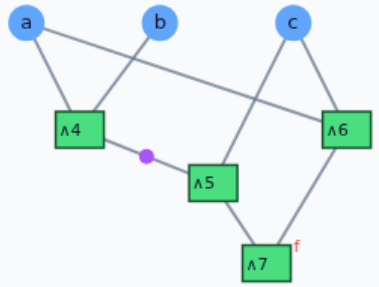
No dup

Why

STRASH

Why

Step 5 / 5 · why · module02-01-aig-strash



Explanation

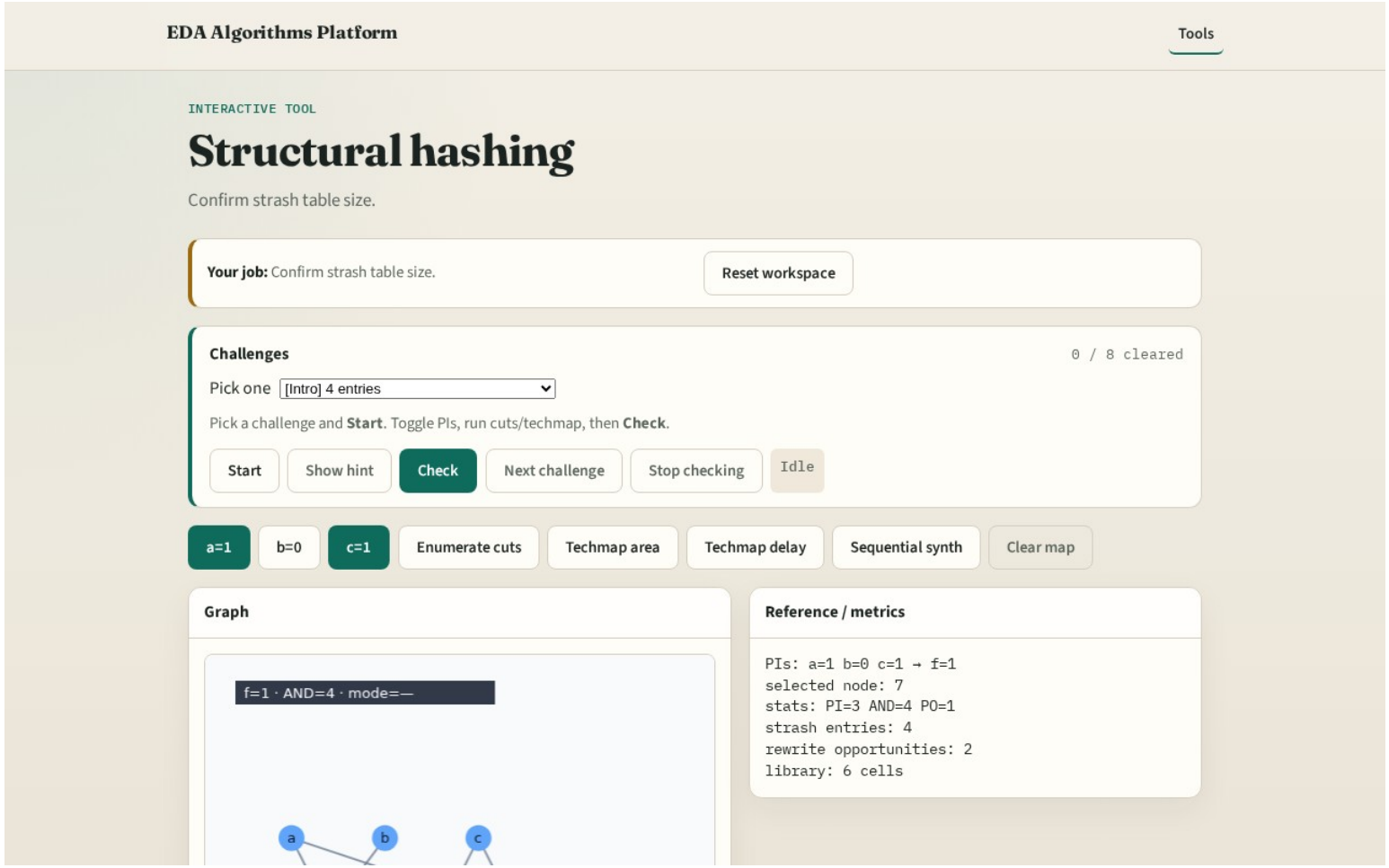
Strash is the first cleanup before rewrite.

- Compact AIG

```
Next rewrite
```

Why

Browser lab track



Browser lab starter

Implement track

- Call build_strash_table; assert len==4

Pitfalls

- Forgetting to sort fanins
- Hashing node id instead of literals

Your turn

- Next: local rewrite opportunities